# Case Study 4

## RAG & Search

**"Find the Evidence That Matters"**

Labs 7-8 — Weeks 7-8 — **Complete v3 with CS1-CS3 Integration**

**Course:** Big Data and Intelligent Analytics
**Term:** Spring 2026

**Designed by**
**Sri Krishnamurthy, CFA**
sri@quantuniversity.com
**Copyright © 2025 QuantUniversity LLC. All Rights Reserved.**

**Assigned:** February 20, 2026
**Due:** March 13, 2026 (3:59 PM)

**"Why did TechCorp score 72 on Data Infrastructure?"**

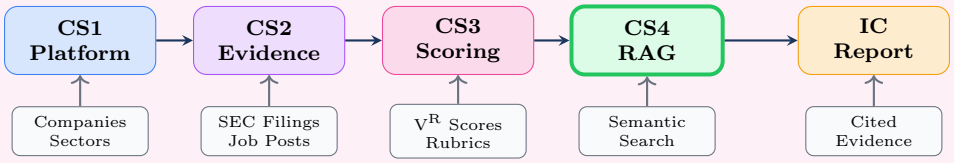
This case study builds the **complete evidence pipeline** from CS1 companies through CS2 evidence to CS3 scores — enabling investment committees to drill down from any score to the specific evidence that justifies it.

**NEW: 8 Integration Tasks connecting CS1→CS2→CS3→CS4**

| | |
|---|---|
| **Weight** | 8% of final grade (100 base + 20 bonus points) |
| **Format** | Individual work |
| **Core Tech** | LiteLLM, ChromaDB, BM25, FastAPI, Redis |
| **Integration** | CS1 Client, CS2 Schema, CS3 Rubrics, Dimension Mapper |
| **PE Workflows** | Score Justification, IC Meeting Prep, Analyst Notes |

**Building on CS1-CS3**

**The Complete Pipeline You Will Build:**




**What CS4 Consumes:**

- CS1: Company metadata (ticker, sector, market_cap_percentile)
- CS2: Document chunks with source_type, signal_category, confidence
- CS3: Dimension scores, rubric criteria, keywords

**What CS4 Produces:** Score justifications with cited evidence for investment committees.

# Contents


**I Lab 7: Foundation & Integration Layer** 4

**1 Lab 7 Preamble** 4
1.1 Learning Objectives . . . 4
1.2 Tools Introduced . . . 4

**2 Task 7.0a: CS1 Company Service Integration [INTEGRATION]** 5
2.1 CS1 Company Schema . . . 5

**3 Task 7.0b: CS2 Evidence Schema & Loader [INTEGRATION]** 7
3.1 CS2 Evidence Schema . . . 7

**4 Task 7.0c: CS3 Scoring API Integration [INTEGRATION]** 10
4.1 CS3 Scoring Schema . . . 10

**5 Task 7.1: Multi-Provider LLM Router** 13

**6 Task 7.2: Semantic Search with Metadata** 15

**II Lab 8: Hybrid RAG & PE Workflows** 17

**7 Lab 8 Preamble** 17
7.1 Learning Objectives . . . 17

**8 Task 8.0a: Evidence-to-Dimension Mapper [INTEGRATION]** 18

**9 Task 8.1: Hybrid Retrieval with RRF Fusion** 20

**10 Task 8.0b: Score Justification Generator [INTEGRATION]** 22

**11 Task 8.0c: IC Meeting Prep Workflow [INTEGRATION]** 25

**12 Task 8.0d: Analyst Notes Collector [INTEGRATION]** 28

**13 Practical Exercise: Complete Pipeline** 30

**III Production Extensions** 32

**14 Extension A: Airflow Evidence Pipeline (+5 pts)** 32

**15 Extension B: Docker Compose (+5 pts)** 33

**16 Evaluation Criteria** 34

**17 Submission** 35


Copyright © 2025 QuantUniversity LLC 

# Part I

# Lab 7: Foundation & Integration Layer

## 1 Lab 7 Preamble

| | |
|---|---|
| **Date** | February 20, 2026 (Thursday) |
| **Focus** | CS1/CS2/CS3 Integration, LLM Routing, Semantic Search |
| **Time Estimate** | Lecture: 2.5 hrs — Lab: 7 hrs — Challenge: +3 hrs |

### 1.1 Learning Objectives

**Learning Objectives**

| Bloom's Level | Objective |
|---|---|
| **Remember** | List the data produced by CS1, CS2, and CS3 |
| **Understand** | Explain how CS2 signals map to CS3 dimensions |
| **Apply** | Implement API clients for CS1, CS2, and CS3 services |
| **Analyze** | Compare evidence quality by source type and confidence |
| **Evaluate** | Assess which evidence best supports a dimension score |
| **Create** | Design the complete evidence indexing pipeline |

### 1.2 Tools Introduced

**Tools Introduced**

| Tool | Purpose | Why This Tool |
|---|---|---|
| **LiteLLM** | Multi-provider LLM routing | 100+ providers, unified API, automatic fallbacks |
| Sentence Transformers | Text embeddings | High-quality 384-dim vectors |
| ChromaDB | Vector database | Simple, persistent, metadata filtering |
| **httpx** | Async HTTP client | Connect to CS1/CS2/CS3 APIs |
| **SSE-Starlette** | Server-Sent Events | Real-time streaming responses |

Copyright © 2025 QuantUniversity LLC 

# 2 Task 7.0a: CS1 Company Service Integration [INTEGRATION]

**Integration Task:**

Connect to CS1 Platform API CS1 provides company metadata needed for:

- Filtering searches by portfolio companies
- Getting sector for position_factor calculation
- Fetching market_cap_percentile for peer comparison

## 2.1 CS1 Company Schema

services/integration/cs1_client.py — CS1 Platform Client

```python
"""CS1 Platform API client for company data."""
import httpx
from dataclasses import dataclass
from typing import List, Optional
from enum import Enum
import structlog

logger = structlog.get_logger()

class Sector(str, Enum):
    TECHNOLOGY = "technology"
    FINANCIAL_SERVICES = "financial_services"
    HEALTHCARE = "healthcare"
    MANUFACTURING = "manufacturing"
    RETAIL = "retail"
    BUSINESS_SERVICES = "business_services"
    CONSUMER = "consumer"

@dataclass
class Company:
    """Company from CS1 Platform."""
    company_id: str
    ticker: str
    name: str
    sector: Sector
    sub_sector: str
    market_cap_percentile: float  # 0-1, for position_factor
    revenue_millions: float
    employee_count: int
    fiscal_year_end: str  # "December", "March", etc.

@dataclass
class Portfolio:
    """PE portfolio from CS1."""
    portfolio_id: str
    name: str
    company_ids: List[str]
    fund_vintage: int

class CS1Client:
    """Client for CS1 Platform API."""

    def __init__(self, base_url: str = "http://localhost:8000"):
        self.base_url = base_url
        self.client = httpx.AsyncClient(timeout=30.0)

    async def get_company(self, ticker: str) -> Company:
        """Fetch company by ticker symbol."""
        response = await self.client.get(
            f"{self.base_url}/api/v1/companies/{ticker}"
        )
        response.raise_for_status()
```

```python
        data = response.json()
        return Company(
            company_id=data["company_id"],
            ticker=data["ticker"],
            name=data["name"],
            sector=Sector(data["sector"]),
            sub_sector=data["sub_sector"],
            market_cap_percentile=data["market_cap_percentile"],
            revenue_millions=data["revenue_millions"],
            employee_count=data["employee_count"],
            fiscal_year_end=data["fiscal_year_end"],
        )

    async def list_companies(
        self,
        sector: Optional[Sector] = None,
        min_revenue: Optional[float] = None,
    ) -> List[Company]:
        """List companies with optional filters."""
        params = {}
        if sector:
            params["sector"] = sector.value
        if min_revenue:
            params["min_revenue"] = min_revenue

        response = await self.client.get(
            f"{self.base_url}/api/v1/companies",
            params=params
        )
        response.raise_for_status()
        return [Company(**c) for c in response.json()]

    async def get_portfolio_companies(
        self,
        portfolio_id: str
    ) -> List[Company]:
        """Get all companies in a PE portfolio."""
        response = await self.client.get(
            f"{self.base_url}/api/v1/portfolios/{portfolio_id}/companies"
        )
        response.raise_for_status()
        return [Company(**c) for c in response.json()]

    async def close(self):
        await self.client.aclose()
```

Copyright © 2025 QuantUniversity LLC 

# 3 Task 7.0b: CS2 Evidence Schema & Loader [INTEGRATION]

**Integration Task:**

Connect to CS2 Evidence Collection CS2 produces evidence with rich metadata that CS4 must preserve:

- **source_type**: SEC 10-K Item 1, Item 1A, Item 7, job posting, etc.
- **signal_category**: technology_hiring, innovation_activity, etc.
- **confidence**: Extraction confidence (0-1)
- **extracted_entities**: Structured entities from the text

## 3.1 CS2 Evidence Schema

services/integration/cs2_client.py — CS2 Evidence Client

```python
"""CS2 Evidence Collection API client."""
from dataclasses import dataclass, field
from typing import List, Optional, Dict, Any
from datetime import datetime
from enum import Enum
import httpx

class SourceType(str, Enum):
    """Evidence source types from CS2."""
    SEC_10K_ITEM_1 = "sec_10k_item_1"      # Business description
    SEC_10K_ITEM_1A = "sec_10k_item_1a"    # Risk factors
    SEC_10K_ITEM_7 = "sec_10k_item_7"      # MD&A
    JOB_POSTING_LINKEDIN = "job_posting_linkedin"
    JOB_POSTING_INDEED = "job_posting_indeed"
    PATENT_USPTO = "patent_uspto"
    PRESS_RELEASE = "press_release"
    GLASSDOOR_REVIEW = "glassdoor_review"      # From CS3 Task 5.0c
    BOARD_PROXY_DEF14A = "board_proxy_def14a"  # From CS3 Task 5.0d
    ANALYST_INTERVIEW = "analyst_interview"    # NEW: DD interviews
    DD_DATA_ROOM = "dd_data_room"              # NEW: Data room docs

class SignalCategory(str, Enum):
    """Signal categories from CS2 collectors."""
    TECHNOLOGY_HIRING = "technology_hiring"
    INNOVATION_ACTIVITY = "innovation_activity"
    DIGITAL_PRESENCE = "digital_presence"
    LEADERSHIP_SIGNALS = "leadership_signals"
    CULTURE_SIGNALS = "culture_signals"
    GOVERNANCE_SIGNALS = "governance_signals"

@dataclass
class ExtractedEntity:
    """Entity extracted from evidence text."""
    entity_type: str  # "ai_investment", "technology", "person", etc.
    text: str
    char_start: int
    char_end: int
    confidence: float
    attributes: Dict[str, Any] = field(default_factory=dict)

@dataclass
class CS2Evidence:
    """Evidence item from CS2 Evidence Collection."""
    evidence_id: str
    company_id: str
    source_type: SourceType
    signal_category: SignalCategory
    content: str
    extracted_at: datetime
```

Copyright © 2025 QuantUniversity LLC 

```python
    confidence: float

    # Optional metadata
    fiscal_year: Optional[int] = None
    source_url: Optional[str] = None
    page_number: Optional[int] = None
    extracted_entities: List[ExtractedEntity] = field(default_factory=list)

    # Indexing status
    indexed_in_cs4: bool = False
    indexed_at: Optional[datetime] = None

class CS2Client:
    """Client for CS2 Evidence Collection API."""

    def __init__(self, base_url: str = "http://localhost:8001"):
        self.base_url = base_url
        self.client = httpx.AsyncClient(timeout=60.0)

    async def get_evidence(
        self,
        company_id: str,
        source_types: Optional[List[SourceType]] = None,
        signal_categories: Optional[List[SignalCategory]] = None,
        min_confidence: float = 0.0,
        indexed: Optional[bool] = None,
        since: Optional[datetime] = None,
    ) -> List[CS2Evidence]:
        """
        Fetch evidence for a company with filters.

        Args:
            company_id: Company ticker or ID
            source_types: Filter by source types
            signal_categories: Filter by signal categories
            min_confidence: Minimum confidence threshold
            indexed: Filter by indexing status (None=all)
            since: Only evidence extracted after this date
        """
        params = {"company_id": company_id}
        if source_types:
            params["source_types"] = ",".join(s.value for s in source_types)
        if signal_categories:
            params["signal_categories"] = ",".join(s.value for s in signal_categories)
        if min_confidence > 0:
            params["min_confidence"] = min_confidence
        if indexed is not None:
            params["indexed"] = indexed
        if since:
            params["since"] = since.isoformat()

        response = await self.client.get(
            f"{self.base_url}/api/v1/evidence",
            params=params
        )
        response.raise_for_status()

        evidence_list = []
        for e in response.json():
            entities = [ExtractedEntity(**ent) for ent in e.get("extracted_entities", [])]
            evidence_list.append(CS2Evidence(
                evidence_id=e["evidence_id"],
                company_id=e["company_id"],
                source_type=SourceType(e["source_type"]),
                signal_category=SignalCategory(e["signal_category"]),
                content=e["content"],
                extracted_at=datetime.fromisoformat(e["extracted_at"]),
                confidence=e["confidence"],
                fiscal_year=e.get("fiscal_year"),
```

```
                source_url=e.get("source_url"),
                page_number=e.get("page_number"),
                extracted_entities=entities,
                indexed_in_cs4=e.get("indexed_in_cs4", False),
            ))
        return evidence_list

    async def mark_indexed(self, evidence_ids: List[str]) -> int:
        """Mark evidence as indexed in CS4."""
        response = await self.client.post(
            f"{self.base_url}/api/v1/evidence/mark-indexed",
            json={"evidence_ids": evidence_ids}
        )
        response.raise_for_status()
        return response.json()["updated_count"]

    async def close(self):
        await self.client.aclose()
```

# 4 Task 7.0c: CS3 Scoring API Integration [INTEGRATION]

**Integration Task:**

Connect to CS3 Scoring Engine CS3 provides the scores and rubrics needed for justification:

- **Dimension scores**: The actual 0-100 scores to justify
- **Rubric criteria**: What each score level means
- **Rubric keywords**: Terms to search for in evidence

## 4.1 CS3 Scoring Schema

services/integration/cs3_client.py — CS3 Scoring Client

```python
"""CS3 Scoring Engine API client."""
from dataclasses import dataclass, field
from typing import List, Dict, Optional, Tuple
from enum import Enum
import httpx

class Dimension(str, Enum):
    """The 7 V^R dimensions from CS3."""
    DATA_INFRASTRUCTURE = "data_infrastructure"
    AI_GOVERNANCE = "ai_governance"
    TECHNOLOGY_STACK = "technology_stack"
    TALENT = "talent"
    LEADERSHIP = "leadership"
    USE_CASE_PORTFOLIO = "use_case_portfolio"
    CULTURE = "culture"

class ScoreLevel(int, Enum):
    """Score levels with ranges."""
    LEVEL_5 = 5  # 80-100: Excellent
    LEVEL_4 = 4  # 60-79: Good
    LEVEL_3 = 3  # 40-59: Adequate
    LEVEL_2 = 2  # 20-39: Developing
    LEVEL_1 = 1  # 0-19: Nascent

    @property
    def name_label(self) -> str:
        labels = {5: "Excellent", 4: "Good", 3: "Adequate",
                  2: "Developing", 1: "Nascent"}
        return labels[self.value]

    @property
    def score_range(self) -> Tuple[int, int]:
        ranges = {5: (80, 100), 4: (60, 79), 3: (40, 59),
                  2: (20, 39), 1: (0, 19)}
        return ranges[self.value]

@dataclass
class DimensionScore:
    """Single dimension score from CS3."""
    dimension: Dimension
    score: float
    level: ScoreLevel
    confidence_interval: Tuple[float, float]  # (lower, upper)
    evidence_count: int
    last_updated: str

@dataclass
class RubricCriteria:
    """Rubric criteria for a dimension level."""
    dimension: Dimension
    level: ScoreLevel
    criteria_text: str
```

```python
    keywords: List[str]
    quantitative_thresholds: Dict[str, float]

    # Example thresholds:
    # {"ai_job_ratio": 0.25, "data_quality_pct": 70}

@dataclass
class CompanyAssessment:
    """Full company assessment from CS3."""
    company_id: str
    assessment_date: str

    # Composite scores
    vr_score: float
    hr_score: float
    synergy_score: float
    org_air_score: float

    # Confidence interval
    confidence_interval: Tuple[float, float]

    # Component scores
    dimension_scores: Dict[Dimension, DimensionScore]

    # Risk adjustments
    talent_concentration: float
    position_factor: float

class CS3Client:
    """Client for CS3 Scoring Engine API."""

    def __init__(self, base_url: str = "http://localhost:8002"):
        self.base_url = base_url
        self.client = httpx.AsyncClient(timeout=30.0)

    async def get_assessment(self, company_id: str) -> CompanyAssessment:
        """Fetch complete assessment for a company."""
        response = await self.client.get(
            f"{self.base_url}/api/v1/assessments/{company_id}"
        )
        response.raise_for_status()
        data = response.json()

        dim_scores = {}
        for dim_name, score_data in data["dimension_scores"].items():
            dim = Dimension(dim_name)
            dim_scores[dim] = DimensionScore(
                dimension=dim,
                score=score_data["score"],
                level=ScoreLevel(score_data["level"]),
                confidence_interval=tuple(score_data["confidence_interval"]),
                evidence_count=score_data["evidence_count"],
                last_updated=score_data["last_updated"],
            )

        return CompanyAssessment(
            company_id=data["company_id"],
            assessment_date=data["assessment_date"],
            vr_score=data["vr_score"],
            hr_score=data["hr_score"],
            synergy_score=data["synergy_score"],
            org_air_score=data["org_air_score"],
            confidence_interval=tuple(data["confidence_interval"]),
            dimension_scores=dim_scores,
            talent_concentration=data["talent_concentration"],
            position_factor=data["position_factor"],
        )

    async def get_dimension_score(
```

```python
        self,
        company_id: str,
        dimension: Dimension
    ) -> DimensionScore:
        """Fetch single dimension score."""
        response = await self.client.get(
            f"{self.base_url}/api/v1/assessments/{company_id}/dimensions/{dimension.value}"
        )
        response.raise_for_status()
        data = response.json()
        return DimensionScore(
            dimension=dimension,
            score=data["score"],
            level=ScoreLevel(data["level"]),
            confidence_interval=tuple(data["confidence_interval"]),
            evidence_count=data["evidence_count"],
            last_updated=data["last_updated"],
        )

    async def get_rubric(
        self,
        dimension: Dimension,
        level: Optional[ScoreLevel] = None
    ) -> List[RubricCriteria]:
        """
        Fetch rubric criteria for a dimension.

        Args:
            dimension: Which dimension
            level: Specific level (None = all levels)
        """
        params = {}
        if level:
            params["level"] = level.value

        response = await self.client.get(
            f"{self.base_url}/api/v1/rubrics/{dimension.value}",
            params=params
        )
        response.raise_for_status()

        return [
            RubricCriteria(
                dimension=dimension,
                level=ScoreLevel(r["level"]),
                criteria_text=r["criteria_text"],
                keywords=r["keywords"],
                quantitative_thresholds=r.get("quantitative_thresholds", {}),
            )
            for r in response.json()
        ]

    async def close(self):
        await self.client.aclose()
```

# 5 Task 7.1: Multi-Provider LLM Router

**LiteLLM:**

Why Multi-Provider? Single-provider dependency creates risk. **LiteLLM** provides unified API to 100+ providers with automatic fallbacks and cost tracking.

services/llm/router.py — LiteLLM Model Router

```python
"""Multi-model routing with LiteLLM and streaming support."""
from typing import AsyncIterator, Dict, Any, List
from enum import Enum
from dataclasses import dataclass, field
from datetime import date
from decimal import Decimal
import litellm
from litellm import acompletion
import structlog

logger = structlog.get_logger()

class TaskType(str, Enum):
    EVIDENCE_EXTRACTION = "evidence_extraction"
    DIMENSION_SCORING = "dimension_scoring"
    JUSTIFICATION_GENERATION = "justification_generation"
    CHAT_RESPONSE = "chat_response"

@dataclass
class ModelConfig:
    primary: str
    fallbacks: List[str]
    temperature: float
    max_tokens: int
    cost_per_1k_tokens: float

MODEL_ROUTING: Dict[TaskType, ModelConfig] = {
    TaskType.EVIDENCE_EXTRACTION: ModelConfig(
        primary="gpt-4o-2024-08-06",
        fallbacks=["claude-sonnet-4-20250514"],
        temperature=0.3, max_tokens=4000, cost_per_1k_tokens=0.015,
    ),
    TaskType.JUSTIFICATION_GENERATION: ModelConfig(
        primary="claude-sonnet-4-20250514",
        fallbacks=["gpt-4o-2024-08-06"],
        temperature=0.2, max_tokens=2000, cost_per_1k_tokens=0.012,
    ),
    TaskType.CHAT_RESPONSE: ModelConfig(
        primary="claude-haiku-4-5-20251001",
        fallbacks=["gpt-3.5-turbo"],
        temperature=0.7, max_tokens=1000, cost_per_1k_tokens=0.002,
    ),
}

@dataclass
class DailyBudget:
    date: date = field(default_factory=date.today)
    spent_usd: Decimal = Decimal("0")
    limit_usd: Decimal = Decimal("50.00")

    def can_spend(self, amount: Decimal) -> bool:
        if self.date != date.today():
            self.date, self.spent_usd = date.today(), Decimal("0")
        return self.spent_usd + amount <= self.limit_usd

class ModelRouter:
    """Route LLM requests with fallbacks and cost tracking."""

```

Copyright © 2025 QuantUniversity LLC 

```python
    def __init__(self, daily_limit_usd: float = 50.0):
        self.daily_budget = DailyBudget(limit_usd=Decimal(str(daily_limit_usd)))

    async def complete(
        self, task: TaskType, messages: List[Dict[str, str]],
        stream: bool = False, **kwargs
    ) -> Any:
        """Route completion request with fallbacks."""
        config = MODEL_ROUTING[task]
        for model in [config.primary] + config.fallbacks:
            try:
                if stream:
                    return self._stream_complete(model, messages, config)
                response = await acompletion(
                    model=model, messages=messages,
                    temperature=config.temperature, max_tokens=config.max_tokens,
                )
                logger.info("llm_complete", model=model, task=task.value)
                return response
            except Exception as e:
                logger.warning("model_failed", model=model, error=str(e))
        raise RuntimeError("All models failed")

    async def _stream_complete(
        self, model: str, messages: List[Dict], config: ModelConfig
    ) -> AsyncIterator[str]:
        response = await acompletion(
            model=model, messages=messages, stream=True,
            temperature=config.temperature,
        )
        async for chunk in response:
            if chunk.choices[0].delta.content:
                yield chunk.choices[0].delta.content
```

# 6 Task 7.2: Semantic Search with Metadata

services/search/vector_store.py — ChromaDB with Rich Metadata

```python
"""ChromaDB vector store with CS2 evidence metadata."""
import chromadb
from chromadb.config import Settings
from sentence_transformers import SentenceTransformer
from typing import List, Dict, Any, Optional
from dataclasses import dataclass

@dataclass
class SearchResult:
    doc_id: str
    content: str
    score: float
    metadata: Dict[str, Any]

class VectorStore:
    """Vector store preserving CS2 evidence metadata."""

    def __init__(self, persist_dir: str = "./chroma_data"):
        self.client = chromadb.PersistentClient(
            path=persist_dir,
            settings=Settings(anonymized_telemetry=False)
        )
        self.collection = self.client.get_or_create_collection(
            name="pe_evidence",
            metadata={"hnsw:space": "cosine"}
        )
        self.encoder = SentenceTransformer("all-MiniLM-L6-v2")

    def index_cs2_evidence(
        self,
        evidence_list: List['CS2Evidence'],
        dimension_mapper: 'DimensionMapper'
    ) -> int:
        """
        Index CS2 evidence with dimension mapping.

        Preserves: source_type, signal_category, confidence,
                   company_id, dimension (from mapper)
        """
        ids, contents, metadatas = [], [], []

        for e in evidence_list:
            # Get dimension weights from CS3's mapping
            dim_weights = dimension_mapper.get_dimension_weights(e.signal_category)
            primary_dim = dimension_mapper.get_primary_dimension(e.signal_category)

            ids.append(e.evidence_id)
            contents.append(e.content)
            metadatas.append({
                "company_id": e.company_id,
                "source_type": e.source_type.value,
                "signal_category": e.signal_category.value,
                "dimension": primary_dim.value,
                "dimension_weights": str(dim_weights),  # JSON string
                "confidence": e.confidence,
                "fiscal_year": e.fiscal_year or 0,
                "source_url": e.source_url or "",
            })

        embeddings = self.encoder.encode(contents).tolist()
        self.collection.add(
            ids=ids, embeddings=embeddings,
            documents=contents, metadatas=metadatas
        )
        return len(ids)
```

```python

    def search(
        self, query: str, top_k: int = 10,
        company_id: Optional[str] = None,
        dimension: Optional[str] = None,
        source_types: Optional[List[str]] = None,
        min_confidence: float = 0.0,
    ) -> List[SearchResult]:
        """Search with metadata filters."""
        # Build ChromaDB where clause
        where = {}
        if company_id:
            where["company_id"] = company_id
        if dimension:
            where["dimension"] = dimension
        if min_confidence > 0:
            where["confidence"] = {"$gte": min_confidence}

        query_embedding = self.encoder.encode(query).tolist()
        results = self.collection.query(
            query_embeddings=[query_embedding],
            n_results=top_k,
            where=where if where else None
        )

        return [
            SearchResult(
                doc_id=results["ids"][0][i],
                content=results["documents"][0][i],
                score=1 - results["distances"][0][i],
                metadata=results["metadatas"][0][i]
            )
            for i in range(len(results["ids"][0]))
        ]
```

# Part II

# Lab 8: Hybrid RAG & PE Workflows

## 7 Lab 8 Preamble

| | |
|---|---|
| **Date** | February 27, 2026 (Thursday) |
| **Focus** | Hybrid RAG, Score Justification, IC Meeting Prep |
| **Time Estimate** | Lecture: 2 hrs — Lab: 6 hrs — Challenge: +3 hrs |

### 7.1 Learning Objectives

**Learning Objectives**

| Bloom's Level | Objective |
|---|---|
| **Remember** | List the components of a score justification |
| **Understand** | Explain why hybrid search outperforms dense-only |
| **Apply** | Implement RRF fusion and HyDE query enhancement |
| **Analyze** | Compare evidence strength across source types |
| **Evaluate** | Assess justification quality for IC presentation |
| **Create** | Design complete IC meeting prep workflow |

Copyright © 2025 QuantUniversity LLC 

# 8 Task 8.0a: Evidence-to-Dimension Mapper [INTEGRATION]

**Integration Task:**

Apply CS3's Signal-to-Dimension Mapping CS2 produces **signal categories** (technology_hiring, innovation_activity).
CS3 scoring requires **dimension scores** (talent, data_infrastructure).
This mapper bridges the gap using CS3's Task 5.0a mapping table.

services/retrieval/dimension_mapper.py — Signal-to-Dimension Mapping

```python
"""Map CS2 signals to CS3 dimensions for indexing."""
from typing import Dict
from services.integration.cs2_client import SignalCategory
from services.integration.cs3_client import Dimension

# From CS3 Task 5.0a - Signal-to-Dimension Mapping Matrix
SIGNAL_TO_DIMENSION_MAP: Dict[SignalCategory, Dict[Dimension, float]] = {
    SignalCategory.TECHNOLOGY_HIRING: {
        Dimension.TALENT: 0.70,
        Dimension.TECHNOLOGY_STACK: 0.20,
        Dimension.CULTURE: 0.10,
    },
    SignalCategory.INNOVATION_ACTIVITY: {
        Dimension.TECHNOLOGY_STACK: 0.50,
        Dimension.USE_CASE_PORTFOLIO: 0.30,
        Dimension.DATA_INFRASTRUCTURE: 0.20,
    },
    SignalCategory.DIGITAL_PRESENCE: {
        Dimension.DATA_INFRASTRUCTURE: 0.60,
        Dimension.TECHNOLOGY_STACK: 0.40,
    },
    SignalCategory.LEADERSHIP_SIGNALS: {
        Dimension.LEADERSHIP: 0.60,
        Dimension.AI_GOVERNANCE: 0.25,
        Dimension.CULTURE: 0.15,
    },
    SignalCategory.CULTURE_SIGNALS: {
        Dimension.CULTURE: 0.80,
        Dimension.TALENT: 0.10,
        Dimension.LEADERSHIP: 0.10,
    },
    SignalCategory.GOVERNANCE_SIGNALS: {
        Dimension.AI_GOVERNANCE: 0.70,
        Dimension.LEADERSHIP: 0.30,
    },
}

# Source type to signal category mapping for SEC filings
SOURCE_TO_SIGNAL: Dict[str, SignalCategory] = {
    "sec_10k_item_1": SignalCategory.DIGITAL_PRESENCE,
    "sec_10k_item_1a": SignalCategory.GOVERNANCE_SIGNALS,
    "sec_10k_item_7": SignalCategory.LEADERSHIP_SIGNALS,
    "job_posting_linkedin": SignalCategory.TECHNOLOGY_HIRING,
    "job_posting_indeed": SignalCategory.TECHNOLOGY_HIRING,
    "patent_uspto": SignalCategory.INNOVATION_ACTIVITY,
    "glassdoor_review": SignalCategory.CULTURE_SIGNALS,
    "board_proxy_def14a": SignalCategory.GOVERNANCE_SIGNALS,
}

class DimensionMapper:
    """Map CS2 evidence to CS3 dimensions."""

    def get_dimension_weights(
        self, signal_category: SignalCategory
    ) -> Dict[Dimension, float]:
```

```python
        """Get dimension weights for a signal category."""
        return SIGNAL_TO_DIMENSION_MAP.get(signal_category, {
            Dimension.DATA_INFRASTRUCTURE: 1.0  # Default
        })

    def get_primary_dimension(
        self, signal_category: SignalCategory
    ) -> Dimension:
        """Get primary dimension (highest weight)."""
        weights = self.get_dimension_weights(signal_category)
        return max(weights.items(), key=lambda x: x[1])[0]

    def get_all_dimensions_for_evidence(
        self, signal_category: SignalCategory,
        min_weight: float = 0.1
    ) -> Dict[Dimension, float]:
        """Get all dimensions with weight >= threshold."""
        weights = self.get_dimension_weights(signal_category)
        return {d: w for d, w in weights.items() if w >= min_weight}
```

# 9 Task 8.1: Hybrid Retrieval with RRF Fusion

**Hybrid RAG:**

Why Hybrid Search? **Dense** finds semantically similar content. **Sparse** (BM25) finds exact keywords.
**Hybrid** combines both using Reciprocal Rank Fusion:

$$\text{RRF_score}(d) = \sum_{r \in R} \frac{w_r}{k + \text{rank}_r(d)}$$

services/retrieval/hybrid.py — Hybrid Retriever

```python
"""Hybrid retrieval with RRF fusion."""
from typing import List, Dict, Any, Optional
from dataclasses import dataclass
from collections import defaultdict
from rank_bm25 import BM25Okapi
from sentence_transformers import SentenceTransformer
import chromadb

@dataclass
class RetrievedDocument:
    doc_id: str
    content: str
    metadata: Dict[str, Any]
    score: float
    retrieval_method: str

class HybridRetriever:
    """Hybrid retrieval combining dense and sparse search."""

    def __init__(self, dense_weight: float = 0.6, sparse_weight: float = 0.4, rrf_k: int = 60):
        self.dense_weight = dense_weight
        self.sparse_weight = sparse_weight
        self.rrf_k = rrf_k
        self.encoder = SentenceTransformer("all-MiniLM-L6-v2")
        self.chroma = chromadb.PersistentClient(path="./chroma_data")
        self.collection = self.chroma.get_or_create_collection(
            name="pe_evidence", metadata={"hnsw:space": "cosine"}
        )
        self._bm25, self._corpus, self._doc_ids, self._metadata = None, [], [], []

    def index_documents(self, documents: List[Dict[str, Any]]) -> int:
        ids = [d["doc_id"] for d in documents]
        contents = [d["content"] for d in documents]
        metadatas = [d.get("metadata", {}) for d in documents]

        # Dense indexing
        embeddings = self.encoder.encode(contents).tolist()
        self.collection.add(ids=ids, embeddings=embeddings, documents=contents, metadatas=metadatas)

        # Sparse indexing
        self._corpus.extend(contents)
        self._doc_ids.extend(ids)
        self._metadata.extend(metadatas)
        self._bm25 = BM25Okapi([c.lower().split() for c in self._corpus])
        return len(documents)

    async def retrieve(
        self, query: str, k: int = 10,
        filter_metadata: Optional[Dict] = None
    ) -> List[RetrievedDocument]:
        n = k * 3

```

```python
        # Dense retrieval
        qe = self.encoder.encode(query).tolist()
        dr = self.collection.query(query_embeddings=[qe], n_results=n, where=filter_metadata)
        dense = [RetrievedDocument(
            doc_id=dr["ids"][0][i], content=dr["documents"][0][i],
            metadata=dr["metadatas"][0][i], score=1-dr["distances"][0][i], retrieval_method="dense"
        ) for i in range(len(dr["ids"][0]))]

        # Sparse retrieval
        scores = self._bm25.get_scores(query.lower().split())
        top_idx = sorted(range(len(scores)), key=lambda i: scores[i], reverse=True)[:n]
        sparse = [RetrievedDocument(
            doc_id=self._doc_ids[i], content=self._corpus[i],
            metadata=self._metadata[i], score=scores[i], retrieval_method="sparse"
        ) for i in top_idx]

        # RRF Fusion
        return self._rrf_fusion(dense, sparse, k)

    def _rrf_fusion(self, dense: List[RetrievedDocument], sparse: List[RetrievedDocument], k: int):
        scores, doc_map = defaultdict(float), {}
        for rank, doc in enumerate(dense):
            scores[doc.doc_id] += self.dense_weight / (self.rrf_k + rank + 1)
            doc_map[doc.doc_id] = doc
        for rank, doc in enumerate(sparse):
            scores[doc.doc_id] += self.sparse_weight / (self.rrf_k + rank + 1)
            if doc.doc_id not in doc_map:
                doc_map[doc.doc_id] = doc

        sorted_ids = sorted(scores.keys(), key=lambda x: scores[x], reverse=True)[:k]
        return [RetrievedDocument(
            doc_id=did, content=doc_map[did].content, metadata=doc_map[did].metadata,
            score=scores[did], retrieval_method="hybrid"
        ) for did in sorted_ids]
```

## 10 Task 8.0b: Score Justification Generator [INTEGRATION]

**PE Workflow:**

The Core PE Use Case **IC Member:** "Why did TechCorp score 72 on Data Infrastructure?"
**Required Output:**

- Score and level (72 = Level 4 "Good")
- Rubric criteria matched ("hybrid cloud, 70-90% data quality")
- Supporting evidence with citations
- Gaps identified (what's missing for Level 5)

services/justification/generator.py — Score Justification

```python
"""Generate score justifications with cited evidence."""
from dataclasses import dataclass, field
from typing import List, Optional
from services.integration.cs3_client import CS3Client, Dimension, DimensionScore, RubricCriteria, ScoreLevel
from services.retrieval.hybrid import HybridRetriever, RetrievedDocument
from services.llm.router import ModelRouter, TaskType

@dataclass
class CitedEvidence:
    """Evidence with citation details."""
    evidence_id: str
    content: str  # Truncated to 500 chars
    source_type: str
    source_url: Optional[str]
    confidence: float
    matched_keywords: List[str]
    relevance_score: float

@dataclass
class ScoreJustification:
    """Complete score justification for IC presentation."""
    company_id: str
    dimension: Dimension
    score: float
    level: int
    level_name: str
    confidence_interval: tuple

    rubric_criteria: str
    rubric_keywords: List[str]

    supporting_evidence: List[CitedEvidence]
    gaps_identified: List[str]

    generated_summary: str
    evidence_strength: str  # "strong", "moderate", "weak"

JUSTIFICATION_PROMPT = """You are a PE analyst preparing score justification for an investment committee.

COMPANY: {company_id}
DIMENSION: {dimension}
SCORE: {score}/100 (Level {level} - {level_name})

RUBRIC CRITERIA FOR THIS LEVEL:
{rubric_criteria}

RUBRIC KEYWORDS TO MATCH:
{rubric_keywords}

SUPPORTING EVIDENCE FOUND:
{evidence_text}

```

```
Generate a concise IC-ready justification (150-200 words) that:
1. States why the score matches this rubric level
2. Cites specific evidence with source references [Source Type, FY]
3. Identifies gaps preventing a higher score
4. Assesses evidence strength (strong/moderate/weak)

Write in professional PE memo style."""

class JustificationGenerator:
    """Generate score justifications."""

    def __init__(self):
        self.cs3 = CS3Client()
        self.retriever = HybridRetriever()
        self.router = ModelRouter()

    async def generate_justification(
        self, company_id: str, dimension: Dimension
    ) -> ScoreJustification:
        """Generate full justification for a dimension score."""

        # 1. Fetch score from CS3
        score = await self.cs3.get_dimension_score(company_id, dimension)

        # 2. Get rubric for that level
        rubrics = await self.cs3.get_rubric(dimension, score.level)
        rubric = rubrics[0] if rubrics else None

        # 3. Build search query from rubric keywords
        query = " ".join(rubric.keywords[:5]) if rubric else dimension.value.replace("_", " ")

        # 4. Search for evidence
        results = await self.retriever.retrieve(
            query=query, k=15,
            filter_metadata={"company_id": company_id, "dimension": dimension.value}
        )

        # 5. Match evidence to rubric keywords
        cited = self._match_to_rubric(results, rubric)

        # 6. Identify gaps (next level criteria not found)
        gaps = await self._identify_gaps(company_id, dimension, score.level, results)

        # 7. Generate summary with LLM
        evidence_text = "\n".join([
            f"[{e.source_type}, conf={e.confidence:.2f}] {e.content[:300]}..."
            for e in cited[:5]
        ])

        response = await self.router.complete(
            task=TaskType.JUSTIFICATION_GENERATION,
            messages=[{"role": "user", "content": JUSTIFICATION_PROMPT.format(
                company_id=company_id,
                dimension=dimension.value.replace("_", " ").title(),
                score=score.score,
                level=score.level.value,
                level_name=score.level.name_label,
                rubric_criteria=rubric.criteria_text if rubric else "N/A",
                rubric_keywords=", ".join(rubric.keywords) if rubric else "N/A",
                evidence_text=evidence_text,
            )}]
        )

        summary = response.choices[0].message.content
        strength = self._assess_strength(cited)

        return ScoreJustification(
            company_id=company_id,
            dimension=dimension,
```

```
            score=score.score,
            level=score.level.value,
            level_name=score.level.name_label,
            confidence_interval=score.confidence_interval,
            rubric_criteria=rubric.criteria_text if rubric else "",
            rubric_keywords=rubric.keywords if rubric else [],
            supporting_evidence=cited,
            gaps_identified=gaps,
            generated_summary=summary,
            evidence_strength=strength,
        )

    def _match_to_rubric(
        self, results: List[RetrievedDocument], rubric: Optional[RubricCriteria]
    ) -> List[CitedEvidence]:
        if not rubric:
            return []

        cited = []
        for r in results:
            matched = [kw for kw in rubric.keywords if kw.lower() in r.content.lower()]
            if matched or r.score > 0.7:
                cited.append(CitedEvidence(
                    evidence_id=r.doc_id,
                    content=r.content[:500],
                    source_type=r.metadata.get("source_type", "unknown"),
                    source_url=r.metadata.get("source_url"),
                    confidence=r.metadata.get("confidence", 0.5),
                    matched_keywords=matched,
                    relevance_score=r.score,
                ))
        return sorted(cited, key=lambda x: len(x.matched_keywords), reverse=True)[:5]

    async def _identify_gaps(
        self, company_id: str, dimension: Dimension,
        current_level: ScoreLevel, evidence: List[RetrievedDocument]
    ) -> List[str]:
        """Find criteria from next level not matched in evidence."""
        if current_level == ScoreLevel.LEVEL_5:
            return []  # Already at top

        next_level = ScoreLevel(current_level.value + 1)
        next_rubrics = await self.cs3.get_rubric(dimension, next_level)
        if not next_rubrics:
            return []

        next_rubric = next_rubrics[0]
        evidence_text = " ".join([e.content.lower() for e in evidence])

        gaps = []
        for kw in next_rubric.keywords:
            if kw.lower() not in evidence_text:
                gaps.append(f"No evidence of '{kw}' (Level {next_level.value} criterion)")

        return gaps[:5]

    def _assess_strength(self, evidence: List[CitedEvidence]) -> str:
        if not evidence:
            return "weak"
        avg_conf = sum(e.confidence for e in evidence) / len(evidence)
        avg_matches = sum(len(e.matched_keywords) for e in evidence) / len(evidence)

        if avg_conf >= 0.8 and avg_matches >= 2:
            return "strong"
        elif avg_conf >= 0.6 or avg_matches >= 1:
            return "moderate"
        return "weak"
```

## 11 Task 8.0c: IC Meeting Prep Workflow [INTEGRATION]

**PE Workflow:**

Investment Committee Preparation Before presenting to IC, analysts need a complete evidence package for all 7 dimensions. This workflow automates that preparation.

services/workflows/ic_prep.py — IC Meeting Preparation

```python
"""Investment Committee meeting preparation workflow."""
from dataclasses import dataclass
from typing import List, Dict
from datetime import datetime
from services.justification.generator import JustificationGenerator, ScoreJustification
from services.integration.cs1_client import CS1Client, Company
from services.integration.cs3_client import CS3Client, Dimension, CompanyAssessment

@dataclass
class ICMeetingPackage:
    """Complete IC meeting preparation package."""
    company: Company
    assessment: CompanyAssessment
    dimension_justifications: Dict[Dimension, ScoreJustification]

    executive_summary: str
    key_strengths: List[str]
    key_gaps: List[str]
    risk_factors: List[str]
    recommendation: str

    generated_at: str
    total_evidence_count: int
    avg_evidence_strength: str

class ICPrepWorkflow:
    """Prepare complete IC meeting evidence package."""

    def __init__(self):
        self.cs1 = CS1Client()
        self.cs3 = CS3Client()
        self.generator = JustificationGenerator()

    async def prepare_meeting(
        self, company_id: str,
        focus_dimensions: List[Dimension] = None,
    ) -> ICMeetingPackage:
        """Generate complete IC meeting package."""
        if focus_dimensions is None:
            focus_dimensions = list(Dimension)

        # 1. Fetch company from CS1
        company = await self.cs1.get_company(company_id)

        # 2. Fetch assessment from CS3
        assessment = await self.cs3.get_assessment(company_id)

        # 3. Generate justifications for each dimension
        justifications = {}
        for dim in focus_dimensions:
            justifications[dim] = await self.generator.generate_justification(
                company_id=company_id, dimension=dim
            )

        # 4. Synthesize findings
        strengths = self._identify_strengths(assessment, justifications)
        gaps = self._identify_gaps(justifications)
        risks = self._assess_risks(assessment, justifications)
```

```python

        # 5. Generate executive summary
        summary = self._generate_summary(company, assessment, justifications)

        # 6. Generate recommendation
        recommendation = self._generate_recommendation(assessment, strengths, gaps)

        # Calculate stats
        total_evidence = sum(len(j.supporting_evidence) for j in justifications.values())
        strength_scores = {"strong": 3, "moderate": 2, "weak": 1}
        avg_strength_num = sum(
            strength_scores[j.evidence_strength] for j in justifications.values()
        ) / len(justifications)
        avg_strength = "strong" if avg_strength_num >= 2.5 else "moderate" if avg_strength_num >= 1.5 else "weak"

        return ICMeetingPackage(
            company=company,
            assessment=assessment,
            dimension_justifications=justifications,
            executive_summary=summary,
            key_strengths=strengths,
            key_gaps=gaps,
            risk_factors=risks,
            recommendation=recommendation,
            generated_at=datetime.now().isoformat(),
            total_evidence_count=total_evidence,
            avg_evidence_strength=avg_strength,
        )

    def _identify_strengths(
        self, assessment: CompanyAssessment,
        justifications: Dict[Dimension, ScoreJustification]
    ) -> List[str]:
        """Identify top 3 strengths."""
        strengths = []
        for dim, j in justifications.items():
            if j.level >= 4 and j.evidence_strength in ["strong", "moderate"]:
                strengths.append(
                    f"{dim.value.replace('_', ' ').title()}: Level {j.level} ({j.level_name}) "
                    f"- {len(j.supporting_evidence)} evidence items"
                )
        return sorted(strengths, key=lambda x: -int(x.split("Level ")[1][0]))[:3]

    def _identify_gaps(
        self, justifications: Dict[Dimension, ScoreJustification]
    ) -> List[str]:
        """Identify critical gaps across dimensions."""
        gaps = []
        for dim, j in justifications.items():
            if j.level <= 2:
                gaps.append(f"{dim.value.replace('_', ' ').title()}: Level {j.level} - needs investment")
            gaps.extend(j.gaps_identified[:2])
        return gaps[:5]

    def _assess_risks(
        self, assessment: CompanyAssessment,
        justifications: Dict[Dimension, ScoreJustification]
    ) -> List[str]:
        """Assess execution risks."""
        risks = []
        if assessment.talent_concentration > 0.25:
            risks.append(f"High talent concentration ({assessment.talent_concentration:.2f}) - key person risk")

        weak_dims = [dim for dim, j in justifications.items() if j.evidence_strength == "weak"]
        if weak_dims:
            risks.append(f"Weak evidence for: {', '.join(d.value for d in weak_dims)}")

```

```python
        if assessment.position_factor < 0:
            risks.append(f"Below-average sector position (factor={assessment.position_factor:.2f})")

        return risks[:5]

    def _generate_summary(
        self, company: Company, assessment: CompanyAssessment,
        justifications: Dict[Dimension, ScoreJustification]
    ) -> str:
        """Generate executive summary paragraph."""
        strong_dims = [d for d, j in justifications.items() if j.level >= 4]
        weak_dims = [d for d, j in justifications.items() if j.level <= 2]

        return (
            f"{company.name} ({company.ticker}) scores {assessment.org_air_score:.0f} on Org-AI-R "
            f"({assessment.confidence_interval[0]:.0f}-{assessment.confidence_interval[1]:.0f} 95% CI). "
            f"V^R={assessment.vr_score:.0f}, H^R={assessment.hr_score:.0f}. "
            f"Strengths in {', '.join(d.value.replace('_', ' ') for d in strong_dims[:2])}. "
            + (f"Gaps in {', '.join(d.value.replace('_', ' ') for d in weak_dims[:2])}. " if weak_dims else "")
            + f"Position factor: {assessment.position_factor:+.2f} vs sector peers."
        )

    def _generate_recommendation(
        self, assessment: CompanyAssessment,
        strengths: List[str], gaps: List[str]
    ) -> str:
        """Generate investment recommendation."""
        if assessment.org_air_score >= 70 and len(strengths) >= 2:
            return "PROCEED - Strong AI readiness with solid evidence base"
        elif assessment.org_air_score >= 50:
            return "PROCEED WITH CAUTION - Moderate AI readiness, gaps addressable"
        else:
            return "FURTHER DILIGENCE - Significant AI capability gaps identified"
```

## 12 Task 8.0d: Analyst Notes Collector [INTEGRATION]

**Integration Task:**

Primary Source Evidence Post-LOI due diligence produces high-confidence evidence:

- Interview transcripts (CTO, CDO, CFO)
- Management meeting notes
- Site visit observations
- Data room document summaries

This collector indexes analyst-generated evidence.

services/collection/analyst_notes.py — Analyst Notes Collector

```python
"""Collect and index analyst notes and interview transcripts."""
from dataclasses import dataclass, field
from typing import List, Optional
from datetime import datetime
from enum import Enum
from services.retrieval.hybrid import HybridRetriever
from services.retrieval.dimension_mapper import DimensionMapper

class NoteType(str, Enum):
    INTERVIEW_TRANSCRIPT = "interview_transcript"
    MANAGEMENT_MEETING = "management_meeting"
    SITE_VISIT = "site_visit"
    DD_FINDING = "dd_finding"
    DATA_ROOM_SUMMARY = "data_room_summary"

@dataclass
class AnalystNote:
    """Analyst-generated evidence."""
    note_id: str
    company_id: str
    note_type: NoteType
    title: str
    content: str

    # Interview metadata
    interviewee: Optional[str] = None
    interviewee_title: Optional[str] = None

    # Assessment context
    dimensions_discussed: List[str] = field(default_factory=list)
    key_findings: List[str] = field(default_factory=list)
    risk_flags: List[str] = field(default_factory=list)

    # Provenance
    assessor: str = ""
    created_at: datetime = field(default_factory=datetime.now)
    confidence: float = 1.0  # Primary source = high confidence

class AnalystNotesCollector:
    """API for analysts to submit and index notes."""

    def __init__(self, retriever: HybridRetriever):
        self.retriever = retriever
        self.mapper = DimensionMapper()

    async def submit_interview(
        self, company_id: str, interviewee: str, interviewee_title: str,
        transcript: str, assessor: str, dimensions_discussed: List[str],
    ) -> str:
        """Submit interview transcript for indexing."""
        note_id = f"interview_{company_id}_{datetime.now().strftime('%Y%m%d%H%M%S')}"

```

```python
        # Determine primary dimension
        primary_dim = dimensions_discussed[0] if dimensions_discussed else "leadership"

        # Index the note
        doc = {
            "doc_id": note_id,
            "content": f"Interview: {interviewee_title}\n\n{transcript}",
            "metadata": {
                "company_id": company_id,
                "source_type": NoteType.INTERVIEW_TRANSCRIPT.value,
                "dimension": primary_dim,
                "confidence": 1.0,  # Primary source
                "assessor": assessor,
                "interviewee_title": interviewee_title,
            }
        }
        self.retriever.index_documents([doc])
        return note_id

    async def submit_dd_finding(
        self, company_id: str, title: str, finding: str,
        dimension: str, severity: str, assessor: str,
    ) -> str:
        """Submit due diligence finding."""
        note_id = f"dd_{company_id}_{datetime.now().strftime('%Y%m%d%H%M%S')}"

        doc = {
            "doc_id": note_id,
            "content": f"{title}\n\n{finding}",
            "metadata": {
                "company_id": company_id,
                "source_type": NoteType.DD_FINDING.value,
                "dimension": dimension,
                "confidence": 1.0,
                "assessor": assessor,
                "severity": severity,
            }
        }
        self.retriever.index_documents([doc])
        return note_id

    async def submit_data_room_summary(
        self, company_id: str, document_name: str, summary: str,
        dimension: str, assessor: str,
    ) -> str:
        """Submit data room document summary."""
        note_id = f"dataroom_{company_id}_{datetime.now().strftime('%Y%m%d%H%M%S')}"

        doc = {
            "doc_id": note_id,
            "content": f"Data Room: {document_name}\n\n{summary}",
            "metadata": {
                "company_id": company_id,
                "source_type": NoteType.DATA_ROOM_SUMMARY.value,
                "dimension": dimension,
                "confidence": 1.0,
                "assessor": assessor,
                "document_name": document_name,
            }
        }
        self.retriever.index_documents([doc])
        return note_id
```

## 13 Practical Exercise: Complete Pipeline

**Practical Exercise:**

NVIDIA Data Infrastructure Justification Run the complete pipeline from CS1 company lookup through CS4 score justification.

exercises/complete_pipeline.py — End-to-End Exercise

```python
"""
Complete Pipeline Exercise: Why did NVIDIA score 78 on Data Infrastructure?

Prerequisites: CS1, CS2, CS3 APIs running with NVDA data
"""
import asyncio
from services.integration.cs1_client import CS1Client
from services.integration.cs2_client import CS2Client
from services.integration.cs3_client import CS3Client, Dimension
from services.retrieval.hybrid import HybridRetriever
from services.retrieval.dimension_mapper import DimensionMapper
from services.justification.generator import JustificationGenerator
from services.workflows.ic_prep import ICPrepWorkflow

async def exercise_nvda_justification():
    """Generate score justification for NVIDIA Data Infrastructure."""

    print("="*60)
    print("EXERCISE: NVIDIA Data Infrastructure Score Justification")
    print("="*60)

    # Step 1: Verify company in CS1
    cs1 = CS1Client()
    company = await cs1.get_company("NVDA")
    print(f"\n[CS1] Company: {company.name}")
    print(f"       Sector: {company.sector.value}")
    print(f"       Market Cap Percentile: {company.market_cap_percentile:.2f}")

    # Step 2: Fetch CS3 score
    cs3 = CS3Client()
    score = await cs3.get_dimension_score("NVDA", Dimension.DATA_INFRASTRUCTURE)
    print(f"\n[CS3] Data Infrastructure Score: {score.score:.1f}")
    print(f"       Level: {score.level.value} ({score.level.name_label})")
    print(f"       95% CI: [{score.confidence_interval[0]:.1f}, {score.confidence_interval[1]:.1f}]")

    # Step 3: Get rubric for Level 4
    rubrics = await cs3.get_rubric(Dimension.DATA_INFRASTRUCTURE, score.level)
    rubric = rubrics[0]
    print(f"\n[CS3] Level {score.level.value} Rubric:")
    print(f"       {rubric.criteria_text[:100]}...")
    print(f"       Keywords: {rubric.keywords[:5]}")

    # Step 4: Fetch and index CS2 evidence
    cs2 = CS2Client()
    evidence = await cs2.get_evidence("NVDA")
    print(f"\n[CS2] Total evidence items: {len(evidence)}")

    mapper = DimensionMapper()
    retriever = HybridRetriever()

    docs = []
    for e in evidence:
        dim_weights = mapper.get_dimension_weights(e.signal_category)
        primary_dim = mapper.get_primary_dimension(e.signal_category)
        docs.append({
            "doc_id": e.evidence_id,
            "content": e.content,
            "metadata": {
```

```
                "company_id": e.company_id,
                "source_type": e.source_type.value,
                "dimension": primary_dim.value,
                "confidence": e.confidence,
            }
        })

    retriever.index_documents(docs)
    print(f"[CS4] Indexed {len(docs)} documents")

    # Step 5: Generate justification
    generator = JustificationGenerator()
    justification = await generator.generate_justification("NVDA", Dimension.DATA_INFRASTRUCTURE)

    # Step 6: Display results
    print("\n" + "="*60)
    print("SCORE JUSTIFICATION")
    print("="*60)
    print(f"\nCompany: {company.name} ({company.ticker})")
    print(f"Dimension: Data Infrastructure")
    print(f"Score: {justification.score:.0f}/100 (Level {justification.level} - {justification.level_name})")
    print(f"Confidence: [{justification.confidence_interval[0]:.0f}, {justification.confidence_interval[1]:.0f
    ↪ }]")

    print(f"\nRubric Match:")
    print(f"  {justification.rubric_criteria[:200]}...")

    print(f"\nSupporting Evidence ({len(justification.supporting_evidence)} items):")
    for i, e in enumerate(justification.supporting_evidence, 1):
        print(f"  {i}. [{e.source_type}] (conf={e.confidence:.2f})")
        print(f"     {e.content[:100]}...")
        if e.matched_keywords:
            print(f"     Matched: {e.matched_keywords}")

    print(f"\nGaps Identified:")
    for gap in justification.gaps_identified:
        print(f"  - {gap}")

    print(f"\nEvidence Strength: {justification.evidence_strength.upper()}")

    print(f"\nGenerated Summary:")
    print(f"  {justification.generated_summary}")

    # Cleanup
    await cs1.close()
    await cs2.close()
    await cs3.close()

if __name__ == "__main__":
    asyncio.run(exercise_nvda_justification())
```

# Part III

# Production Extensions

## 14 Extension A: Airflow Evidence Pipeline (+5 pts)

**Apache Airflow:**

Automated Evidence Indexing Nightly pipeline to fetch new CS2 evidence and index in CS4.

dags/evidence_indexing_dag.py — Airflow DAG

```python
"""Automated evidence indexing pipeline."""
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python import PythonOperator
import structlog

logger = structlog.get_logger()

default_args = {
    "owner": "pe-analytics",
    "retries": 3,
    "retry_delay": timedelta(minutes=5),
    "retry_exponential_backoff": True,
}

dag = DAG(
    dag_id="pe_evidence_indexing",
    default_args=default_args,
    schedule_interval="0 2 * * *",  # 2 AM daily
    start_date=datetime(2026, 2, 20),
    catchup=False,
)

def fetch_new_evidence(**context):
    """Fetch unindexed evidence from CS2."""
    import httpx
    response = httpx.get("http://cs2-api:8001/api/v1/evidence", params={"indexed": False})
    evidence = response.json()
    context["ti"].xcom_push(key="evidence", value=evidence)
    return len(evidence)

def index_evidence(**context):
    """Index evidence in vector store."""
    from services.retrieval.hybrid import HybridRetriever
    from services.retrieval.dimension_mapper import DimensionMapper

    evidence = context["ti"].xcom_pull(key="evidence", task_ids="fetch_evidence")
    if not evidence:
        return 0

    mapper = DimensionMapper()
    retriever = HybridRetriever()

    docs = []
    for e in evidence:
        primary_dim = mapper.get_primary_dimension(e["signal_category"])
        docs.append({
            "doc_id": e["evidence_id"],
            "content": e["content"],
            "metadata": {
                "company_id": e["company_id"],
                "source_type": e["source_type"],
```

```
                "dimension": primary_dim,
                "confidence": e["confidence"],
            }
        })

    return retriever.index_documents(docs)

t1 = PythonOperator(task_id="fetch_evidence", python_callable=fetch_new_evidence, dag=dag)
t2 = PythonOperator(task_id="index_evidence", python_callable=index_evidence, dag=dag)
t1 >> t2
```

## 15 Extension B: Docker Compose (+5 pts)

docker-compose.yml

```
version: "3.8"
services:
  rag-api:
    build: .
    ports: ["8000:8000"]
    environment:
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - CHROMA_PERSIST_DIR=/data/chroma
    volumes:
      - chroma_data:/data/chroma
    depends_on: [redis]

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
    volumes: [redis_data:/data]
    command: redis-server --appendonly yes

volumes:
  chroma_data:
    name: pe_rag_chroma
  redis_data:
    name: pe_rag_redis
```

Copyright © 2025 QuantUniversity LLC 

## 16 Evaluation Criteria

**Requirements**

**Core Requirements (100 points)**

**Integration Layer (25 points)** [INTEGRATION]

- ✓ CS1 Company Client (5 pts)
- ✓ CS2 Evidence Schema & Loader (8 pts)
- ✓ CS3 Scoring API Client (7 pts)
- ✓ Dimension Mapper (5 pts)

**RAG Infrastructure (25 points)**

- ✓ LiteLLM multi-provider router (8 pts)
- ✓ Hybrid retrieval (Dense + BM25) (10 pts)
- ✓ HyDE query enhancement (7 pts)

**PE Workflows (30 points)** [INTEGRATION]

- ✓ Score Justification Generator (12 pts)
- ✓ IC Meeting Prep Workflow (10 pts)
- ✓ Analyst Notes Collector (8 pts)

**API & Testing (20 points)**

- ✓ Search API with filters (8 pts)
- ✓ Justification API endpoint (7 pts)
- ✓ Unit & integration tests (5 pts)

**Extensions (+10 bonus points)**

- ✓ Airflow evidence pipeline (+5 pts)
- ✓ Docker Compose deployment (+5 pts)

Copyright © 2025 QuantUniversity LLC

## 17 Submission

**Due:** March 13, 2026 at 3:59 PM

```
cs4_rag_search/
  |-- src/services/
    |-- integration/        # NEW: CS1/CS2/CS3 clients
                            cs1_client.py, cs2_client.py, cs3_client.py
    |-- llm/router.py       # LiteLLM
    |-- retrieval/
                            hybrid.py, dimension_mapper.py, hyde.py
    |-- justification/generator.py # Score justification
    |-- workflows/ic_prep.py# IC meeting prep
    +-- collection/analyst_notes.py# 3Ds evidence
  |-- exercises/complete_pipeline.py
  |-- dags/evidence_indexing_dag.py
  |-- docker-compose.yml
  +-- tests/
```

**"Why did TechCorp score 72 on Data Infrastructure?"**

**Now you can answer with cited evidence.**

**Designed by Sri Krishnamurthy, CFA** · sri@quantuniversity.com

**Copyright © 2025 QuantUniversity LLC. All Rights Reserved.**